

master 4 branches 88 tags

Go to file Code

About

Laravel IDE Helper

#laravel #autocomplete #hacktoberfest

Readme
MIT license
12.7k stars
270 watching
1.1k forks

Releases 35

v2.12.3 Latest
on Mar 6
+ 34 releases

Sponsor this project

barryvdh Barry vd. Heuvel
https://fruitcake.nl

Learn more about GitHub Sponsors

Packages

No packages published

Used by 51.5k

51,470

Contributors 194

+ 183 contributors

Languages

PHP 100.0%

github	Fix integration tests by fixing a PHP 8.1+ deprecation warnings in Re...	6 months ago
config	Implement new config to specify return type of custom relations (#1300)	6 months ago
resources/views	Disabled PHPCS (#1301)	10 months ago
src	Add support for CastsInboundAttributes (#1329)	2 months ago
tests	Add support for CastsInboundAttributes (#1329)	2 months ago
editorconfig	Add test config, CS rules and fix CS	7 years ago
.gitattributes	Update .gitattributes (#1355)	3 months ago
.gitignore	[CODESTYLE] Replace phpcs with php-cs-fixer (#1030)	2 years ago
.php_cs.common.php	Apply "@psr12" in php-cs-fixer (#1150)	2 years ago
.php_cs.dist	[CODESTYLE] Replace phpcs with php-cs-fixer (#1030)	2 years ago
.php_cs.tests.php	[CODESTYLE] Replace phpcs with php-cs-fixer (#1030)	2 years ago
CHANGELOG.md	Add support for CastsInboundAttributes (#1329)	2 months ago
LICENSE.md	Update LICENSE.md	5 years ago
README.md	Fix typo	3 months ago
composer.json	Feat useclassmappackage (#1357)	5 months ago
phpunit.xml.dist	Migrate PHPUnit configuration (#1010)	2 years ago
psalm-baseline.xml	Remove already resolved issue from psalm baseline	2 years ago
psalm.xml	[Chore] Introduce psalm (#962)	2 years ago

README.md

Laravel IDE Helper Generator

Tests passing license MIT stable v2.12.3 downloads 48.94 M Powered By fruitcake

Complete PHPDocs, directly from the source

This package generates helper files that enable your IDE to provide accurate autocomplete. Generation is done based on the files in your project, so they are always up-to-date.

It supports Laravel 8+ and PHP 7.3+

- Installation
- Usage
 - Automatic PHPDoc generation for Laravel Facades
 - Automatic PHPDocs for models
 - Model Directories
 - Ignore Models
 - Model Hooks
 - Automatic PHPDocs generation for Laravel Fluent methods
 - Auto-completion for factory builders
 - PhpStorm Meta for Container instances
- Usage with Lumen
 - Enabling Facades
 - Adding the Service Provider
 - Adding Additional Facades
- License

Installation

Require this package with composer using the following command:

```
composer require --dev barryvdh/laravel-ide-helper
```

This package makes use of [Laravel's package auto-discovery mechanism](#), which means if you don't install dev dependencies in production, it also won't be loaded.

If for some reason you want manually control this:

- add the package to the `extra.laravel.dont-discover` key in `composer.json`, e.g.

```
"extra": {
    "laravel": {
        "dont-discover": [
            "barryvdh/laravel-ide-helper"
        ]
    }
}
```

- Add the following class to the `providers` array in `config/app.php`:

```
Barryvdh\LaravelIdeHelper\IdeHelperServiceProvider::class,
```

If you want to manually load it only in non-production environments, instead you can add this to your `AppServiceProvider` with the `register()` method:

```
public function register()
{
    if ($this->app->isLocal()) {
        $this->app->register(\Barryvdh\LaravelIdeHelper\IdeHelperServiceProvider::class);
    }
    // ...
}
```

Note: Avoid caching the configuration in your development environment, it may cause issues after installing this package; respectively clear the cache beforehand via `php artisan cache:clear` if you encounter problems when running the commands

Usage

Check out this [Laracasts video](#) for a quick introduction/explanation!

- `php artisan ide-helper:generate` - PHPDoc generation for Laravel Facades
- `php artisan ide-helper:models` - PHPDocs for models
- `php artisan ide-helper:meta` - PhpStorm Meta file

Note: You do need CodeComplice for Sublime Text: <https://github.com/spectacles/CodeComplice>

Automatic PHPDoc generation for Laravel Facades

You can now re-generate the docs yourself (for future updates)

```
php artisan ide-helper:generate
```

Note: `bootstrap/compiled.php` has to be cleared first, so run `php artisan clear-compiled` before generating.

This will generate the file `_ide_helper.php` which is expected to be additionally parsed by your IDE for autocomplete. You can use the config `filename` to change its name.

You can configure your `composer.json` to do this each time you update your dependencies:

```
"scripts": {
    "post-update-cmd": [
        "Illuminate\\Foundation\\ComposerScripts::postUpdate",
        "@php artisan ide-helper:generate",
        "@php artisan ide-helper:meta"
    ]
},
```

You can also publish the config file to change implementations (ie. interface to specific class) or set defaults for `--helpers`.

```
php artisan vendor:publish --provider="Barryvdh\\LaravelIdeHelper\\IdeHelperServiceProvider" --tag=config
```

The generator tries to identify the real class, but if it cannot be found, you can define it in the config file.

Some classes need a working database connection. If you do not have a default working connection, some facades will not be included. You can use an in-memory SQLite driver by adding the `-M` option.

You can choose to include helper files. This is not enabled by default, but you can override it with the `--helpers (-H)` option. The `Illuminate\\Support\\helpers.php` is already set up, but you can add/remove your own files in the config file.

Automatic PHPDoc generation for macros and mixins

This package can generate PHPDocs for macros and mixins which will be added to the `_ide_helper.php` file.

But this only works if you use type hinting when declaring a macro.

```
Str::macro('concat', function(string $str1, string $str2) : string {
    return $str1 . $str2;
});
```

Automatic PHPDocs for models

If you don't want to write your properties yourself, you can use the command `php artisan ide-helper:models` to generate PHPDocs, based on table columns, relations and getters/setters.

⚠ Note: this command requires a working database connection to introspect the table of each model

By default, you are asked to overwrite or write to a separate file (`_ide_helper_models.php`). You can write the comments directly to your Model file, using the `--write (-w)` option, or force to not write with `--nowrite (-N)`.

Alternatively using the `--write-mixin (-M)` option will only add a mixin tag to your Model file, writing the rest in (`_ide_helper_models.php`). The class name will be different from the model, avoiding the IDE duplicate annoyance.

⚠ Please make sure to back up your models, before writing the info.

Writing to the models should keep the existing comments and only append new properties/methods. The existing PHPDoc is replaced, or added if not found. With the `--reset (-R)` option, the existing PHPDocs are ignored, and only the newly found columns/relations are saved as PHPDocs.

```
php artisan ide-helper:models "App\\Models\\Post"
```

```
/**
 * App\\Models\\Post
 *
 * @property integer $id
 * @property integer $author_id
 * @property string $title
 * @property string $text
 * @property \\Illuminate\\Support\\Carbon $created_at
 * @property \\Illuminate\\Support\\Carbon $updated_at
 * @property-read \\User $author
 * @property-read \\Illuminate\\Database\\Eloquent\\Collection|\\Comment[] $comments
 * @method static \\Illuminate\\Database\\Eloquent\\Builder|\\App\\Models\\Post newModelQuery()
 * @method static \\Illuminate\\Database\\Eloquent\\Builder|\\App\\Models\\Post newQuery()
 * @method static \\Illuminate\\Database\\Eloquent\\Builder|\\App\\Models\\Post query()
 * @method static \\Illuminate\\Database\\Eloquent\\Builder|\\App\\Models\\Post whereTitle($value)
 * @method static \\Illuminate\\Database\\Eloquent\\Builder|\\App\\Models\\Post forAuthors(\\User ...$authors)
 * ...
 */
```

With the `--write-mixin (-M)` option

```
/**
 * ...
 * @mixin IdeHelperPost
 */
```

Model Directories

By default, models in `app/models` are scanned. The optional argument tells what models to use (also outside `app/models`).

```
php artisan ide-helper:models "App\\Models\\Post" "App\\Models\\User"
```

You can also scan a different directory, using the `--dir` option (relative from the base path):

```
php artisan ide-helper:models --dir="path/to/models" --dir="app/src/Model"
```

You can publish the config file (`php artisan vendor:publish`) and set the default directories.

Ignore Models

Models can be ignored using the `--ignore (-I)` option

```
php artisan ide-helper:models --ignore="App\\Models\\Post,App\\Models\\User"
```

Or can be ignored by setting the `ignored_models` config

```
'ignored_models' => [
    App\\Post::class,
    Api\\User::class
],
```

Magic where* methods

Eloquent allows calling `where<Attribute>` on your models, e.g. `Post::whereTitle(..)` and automatically translates this to e.g. `Post::where('title', '=', '..')`.

If for some reason it's undesired to have them generated (one for each column), you can disable this via config `write_model_magic_where` and setting it to `false`.

Magic `*_count` properties

You may use the `::withCount` method to count the number results from a relationship without actually loading them. Those results are then placed in attributes following the `<columnName>_count` convention.

By default, these attributes are generated in the phpdoc. You can turn them off by setting the config `write_model_relation_count_properties` to `false`.

Support `@comment` based on DocBlock

In order to better support IDEs, relations and getters/setters can also add a comment to a property like table columns. Therefore a custom docblock `@comment` is used:

```
class Users extends Model
{
    /**
     * @comment Get User's full name
     *
     * @return string
     */
    public function getFullNameAttribute(): string
    {
        return $this->first_name . ' ' . $this->last_name ;
    }
}

// => after generate models

/**
 * App\Models\Users
 *
 * @property-read string $full_name Get User's full name
 * ...
 */
```

Dedicated Eloquent Builder methods

A new method to the eloquent models was added called `newEloquentBuilder` [Reference](#) where we can add support for creating a new dedicated class instead of using local scopes in the model itself.

If for some reason it's undesired to have them generated (one for each column), you can disable this via config `write_model_external_builder_methods` and setting it to `false`.

Unsupported or custom database types

Common column types (e.g. varchar, integer) are correctly mapped to PHP types (`string` , `int`).

But sometimes you may want to use custom column types in your database like `geography` , `jsonb` , `citext` , `bit` , etc. which may throw an "Unknown database type"-Exception.

For those special cases, you can map them via the config `custom_db_types`. Example:

```
'custom_db_types' => [
    'mysql' => [
        'geography' => 'array',
        'point' => 'array',
    ],
    'postgresql' => [
        'jsonb' => 'string',
        '_int4' => 'array',
    ],
],
```

Custom Relationship Types

If you are using relationships not built into Laravel you will need to specify the name and returning class in the config to get proper generation.

```
'additional_relation_types' => [
    'externalHasMany' => \My\Package\externalHasMany::class
],
```

Found relationships will typically generate a return value based on the name of the relationship.

If your custom relationships don't follow this traditional naming scheme you can define its return type manually. The available options are `many` and `morphTo`.

```
'additional_relation_return_types' => [
    'externalHasMultiple' => 'many'
],
```

Model Hooks

If you need additional information on your model from sources that are not handled by default, you can hook in to the generation process with model hooks to add extra information on the fly. Simply create a class that implements `ModelHookInterface` and add it to the `model_hooks` array in the config:

```
'model_hooks' => [
    MyCustomHook::class,
],
```

The `run` method will be called during generation for every model and receives the current running `ModelsCommand` and the current `Model`, e.g.:

```
class MyCustomHook implements ModelHookInterface
{
    public function run(ModelsCommand $command, Model $model): void
    {
        if (! $model instanceof MyModel) {
            return;
        }

        $command->setProperty('custom', 'string', true, false, 'My custom property');
        $command->unsetMethod('method');
        $command->setMethod('method', $command->getMethodType($model, '\Some\Class'), ['$param']);
    }
}

/**
 * MyModel
 *
 * @property integer $id
 * @property-read string $custom
```

Automatic PHPDocs generation for Laravel Fluent methods

If you need PHPDocs support for Fluent methods in migration, for example

```
$table->string("somestring")->nullable()->index();
```

After publishing vendor, simply change the `include_fluent` line in your `config/ide-helper.php` file into:

```
'include_fluent' => true,
```

Then run `php artisan ide-helper:generate`, you will now see all Fluent methods recognized by your IDE.

Auto-completion for factory builders

If you would like the `factory()->create()` and `factory()->make()` methods to return the correct model class, you can enable custom factory builders with the `include_factory_builders` line in your `config/ide-helper.php` file. Deprecated for Laravel 8 or latest.

```
'include_factory_builders' => true,
```

For this to work, you must also publish the PhpStorm Meta file (see below).

PhpStorm Meta for Container instances

It's possible to generate a PhpStorm meta file to [add support for factory design pattern](#). For Laravel, this means we can make PhpStorm understand what kind of object we are resolving from the IoC Container. For example, `events` will return an `Illuminate\Events\Dispatcher` object, so with the meta file you can call `app('events')` and it will autocomplete the Dispatcher methods.

```
php artisan ide-helper:meta
```

```
app('events')->fire();
\App::make('events')->fire();

/** @var \Illuminate\Foundation\Application $app */
$app->make('events')->fire();

// When the key is not found, it uses the argument as class name
app('App\SomeClass');
// Also works with
app(App\SomeClass::class);
```

Note: You might need to restart PhpStorm and make sure `.phpstorm.meta.php` is indexed.

Note: When you receive a `FatalException: class not found`, check your config (for example, remove S3 as cloud driver when you don't have S3 configured. Remove Redis ServiceProvider when you don't use it).

You can change the generated filename via the config `meta_filename`. This can be useful for cases where you want to take advantage of PhpStorm's support of the `directory .phpstorm.meta.php/`: all files placed there are parsed, should you want to provide additional files to PhpStorm.

Usage with Lumen

This package is focused on Laravel development, but it can also be used in Lumen with some workarounds. Because Lumen works a little different, as it is like a bare bone version of Laravel and the main configuration parameters are instead located in `bootstrap/app.php`, some alterations must be made.

Enabling Facades

While Laravel IDE Helper can generate automatically default Facades for code hinting, Lumen doesn't come with Facades activated. If you plan in using them, you must enable them under the `Create The Application` section, uncommenting this line:

```
// $app->withFacades();
```

From there, you should be able to use the `create_alias()` function to add additional Facades into your application.

Adding the Service Provider

You can install Laravel IDE Helper in `app/Providers/AppServiceProvider.php`, and uncommenting this line that registers the App Service Providers, so it can properly load.

```
// $app->register(App\Providers\AppServiceProvider::class);
```

If you are not using that line, that is usually handy to manage gracefully multiple Laravel/Lumen installations, you will have to add this line of code under the `Register Service Providers` section of your `bootstrap/app.php`.

```
if ($app->environment() !== 'production') {
    $app->register(\Barryvdh\LaravelIdeHelper\IdeHelperServiceProvider::class);
}
```

After that, Laravel IDE Helper should work correctly. During the generation process, the script may throw exceptions saying that some Class(s) doesn't exist or there are some undefined indexes. This is normal, as Lumen has some default packages stripped away, like Cookies, Storage and Session. If you plan to add these packages, you will have to add them manually and create additional Facades if needed.

Adding Additional Facades

Currently, Lumen IDE Helper doesn't take into account additional Facades created under `bootstrap/app.php` using `create_alias()`, so you need to create a `config/app.php` file and add your custom aliases under an `aliases` array again, like so:

```
return [
    'aliases' => [
        'CustomAliasOne' => Example\Support\Facades\CustomAliasOne::class,
        'CustomAliasTwo' => Example\Support\Facades\CustomAliasTwo::class,
        //...
    ]
];
```

After you run `php artisan ide-helper:generate`, it's recommended (but not mandatory) to rename `config/app.php` to something else, until you have to re-generate the docs or after passing to production environment. Lumen 5.1+ will read this file for configuration parameters if it is present, and may overlap some configurations if it is completely populated.

License

The Laravel IDE Helper Generator is open-sourced software licensed under the [MIT license](#)